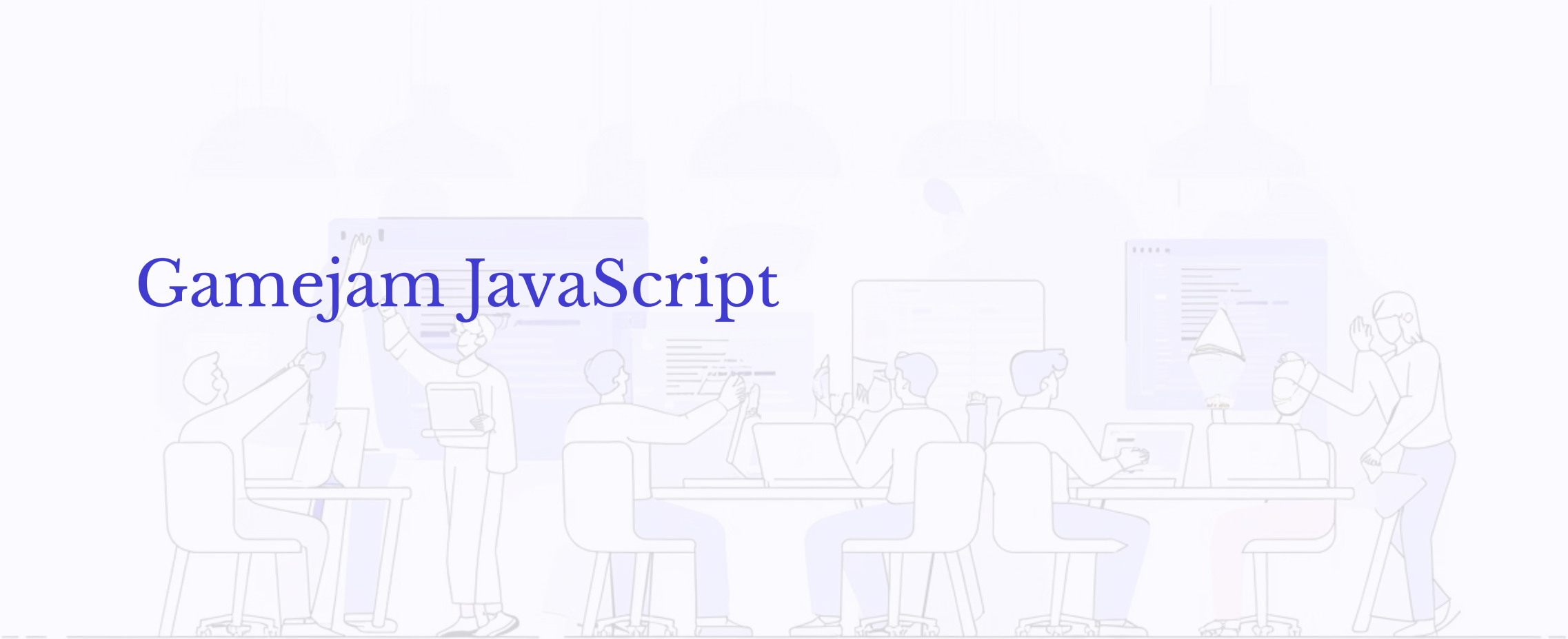
Gamejam JavaScript



Cahier des charges obligatoire

Le premier jour de la gamejam est consacré à la planification et à la structuration de votre projet. Cette étape détermine la réussite de votre développement et assure une collaboration efficace au sein de l'équipe.

Composition de l'équipe

Définissez votre binôme ou trinôme avec les rôles de chacun

Description du projet

Rédigez un pitch clair en 3-4 phrases maximum

Fonctionnalités MVP

Listez les features essentielles pour votre produit minimum viable

Livrables techniques

- Lien du dépôt GitHub configuré
- Maquettes ou wireframes des écrans principaux
- Architecture technique prévisionnelle

Organisation

- Répartition claire des tâches par membre
- Planning prévisionnel des sprints
- Identification des dépendances critiques

Jours de développement et de présentation

Les jours 2 à 5 sont dédiés à la concrétisation de votre projet. C'est le moment d'appliquer les concepts d'architecture et de programmation orientée objet, de gérer les erreurs pour garantir la robustesse du code, et de veiller à une interface utilisateur multi-écrans et un responsive design.

01

Jours 2 à 4 : Développement intensif

Vous plongerez au cœur du code, en utilisant JavaScript pour implémenter les fonctionnalités clés de votre jeu. L'objectif est de bâtir une base solide pour votre projet.

02

Jour 5, début de matinée : Finitions et optimisations

Cette matinée est réservée aux ajustements finaux. Concentrez-vous sur les améliorations qui n'impactent pas la logique métier, telles que le CSS pour l'esthétique, l'optimisation des performances ou de légères retouches d'interface. Aucune modification structurelle du code ne devrait être effectuée à ce stade pour éviter de casser des fonctionnalités.

03

Jour 5, fin de matinée et après-midi : Présentations

À partir de 11h, préparez-vous à présenter le fruit de votre travail.

> Tous les commits à partir de la première présentation seront rejetés pour l'évaluation du projet.

Architecture & programmation orientée objet

La programmation orientée objet est au cœur de cette gamejam. Votre projet doit démontrer une maîtrise des concepts fondamentaux de la POO en JavaScript, notamment l'encapsulation, l'héritage et le polymorphisme. Cette approche vous permettra de créer un code maintenable, réutilisable et évolutif.

01

Classes JavaScript obligatoires

Structurez votre code avec des classes ES6, en utilisant des constructeurs, des méthodes et des propriétés privées quand nécessaire.

02

Minimum 3 héritages

Créez une hiérarchie de classes avec au moins 3 niveaux d'héritage. Exemple : Entity → Player, Enemy, Item.

03

Polymorphisme recommandé

Implémentez des méthodes qui se comportent différemment selon la classe enfant qui les appelle.

> **Exemple d'architecture :** Une classe parente `Entity` peut contenir les propriétés communes (x, y, width, height) tandis que les classes `Player` et `Enemy` héritent et ajoutent leurs propres comportements spécifiques.

Robustesse du code

La gestion des erreurs n'est pas une option mais une exigence fondamentale pour garantir la stabilité de votre application. Vous devez prévoir les cas limites, valider les entrées utilisateur et gérer les défaillances potentielles du système.

Try/Catch dans les opérations critiques

Encapsulez toutes les opérations risquées (parsing JSON, accès DOM, calculs, ...) dans des blocs try/catch pour intercepter les erreurs.

```
try {
  const data = JSON.parse(localStorage.getItem('game'));
} catch(e) {
  console.error('Erreur de parsing:', e);
}
```

Erreurs personnalisées

Créez et levez des erreurs explicites avec `throw new Error()` pour faciliter le débogage et améliorer la traçabilité.

```
if (!username) {
  throw new Error("Nom d'utilisateur requis");
}
```

Interface utilisateur multi-écrans

Votre application doit proposer une expérience utilisateur complète avec plusieurs écrans distincts. Cette approche multi-écrans reflète les standards des applications web modernes et vous permet de structurer logiquement votre interface. Chaque écran doit avoir un rôle spécifique dans le parcours utilisateur et contribuer à une navigation intuitive.

Écran d'accueil/menu

Point d'entrée principal avec navigation claire vers les autres sections

Écrans bonus

Paramètres, scores, game over pour enrichir l'expérience

1 2 3

Écran de jeu principal

Interface principale où se déroule l'action ou la fonctionnalité centrale

Exigences minimales

Vous devez créer au minimum 2 écrans distincts : un écran d'accueil ou menu et un écran de jeu principal. Ces écrans doivent être clairement séparés dans votre code et offrir une navigation fluide entre eux.

Pour améliorer votre projet, envisagez d'ajouter des écrans supplémentaires comme un écran de paramètres permettant de configurer l'expérience, un tableau des scores pour afficher les performances, ou un écran de fin de partie avec statistiques détaillées.

Responsive Design

Le responsive design n'est pas une option mais une obligation dans le développement web moderne. 100% de vos vues doivent s'adapter parfaitement à tous les formats d'écran, du smartphone au desktop. Cette contrainte vous pousse à penser mobile-first et à créer des interfaces véritablement universelles.

Tests multi-supports obligatoires

Testez systématiquement sur mobile, tablette et desktop. Utilisez les outils de développement du navigateur pour simuler différentes résolutions.

- Mobile : 320px à 480px
- Tablette : 768px à 1024px
- Desktop : 1200px et plus

Frameworks CSS autorisés

Vous pouvez utiliser Bootstrap, Tailwind CSS ou tout autre framework CSS pour accélérer le développement responsive.

- Grid systems natifs ou frameworks
- Flexbox pour les layouts flexibles
- Media queries personnalisées

Approche mobile-first

Concevez d'abord pour les petits écrans puis enrichissez progressivement pour les écrans plus larges.

- Navigation hamburger sur mobile
- Touch-friendly pour tactile
- Images optimisées par taille

Stack technique : JavaScript Vanilla

JavaScript vanilla obligatoire

Toute la logique doit être écrite en JavaScript ES6+ pur, sans bibliothèques tierces comme jQuery, React ou Vue. TypeScript est accepté comme alternative typée.

Frameworks CSS autorisés

Pour le styling uniquement, vous pouvez utiliser Bootstrap, Tailwind ou tout framework CSS. Le HTML5 et CSS3 natifs restent la base.

Aucune bibliothèque JS tierce

Interdiction formelle d'utiliser des bibliothèques JavaScript externes (hors PixiJS/Phaser). Cette règle garantit que vous développez vos propres solutions.

Pourquoi JavaScript vanilla ? Cette contrainte vous permet de comprendre en profondeur le DOM, les événements, les promesses et l'asynchrone sans dépendre d'abstractions. C'est une compétence fondamentale qui vous rendra meilleur développeur, quel que soit le framework que vous utiliserez par la suite.

Persistance des données avec localStorage

localStorage obligatoire

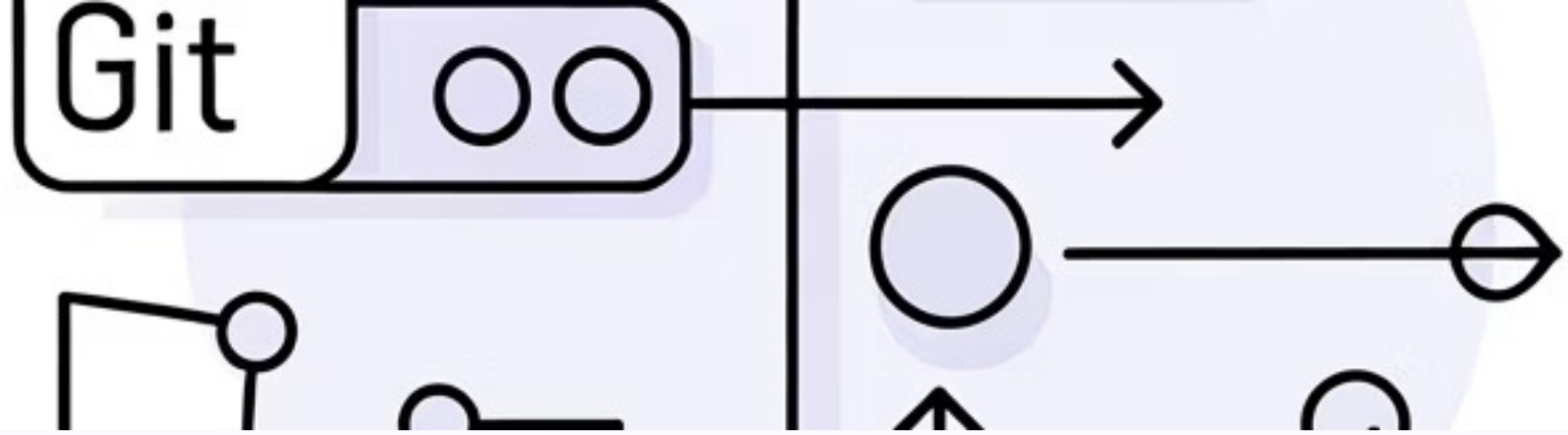
Vous devez utiliser le localStorage pour persister au moins une information :
scores, progression, préférences utilisateur, ou état du jeu.

Cette donnée doit être chargée au démarrage de l'application et mise à jour en temps réel.

Backend optionnel

Un backend n'est pas requis, mais vous pouvez utiliser json-server pour simuler une API REST si votre projet le nécessite. Aucune base de données externe n'est obligatoire.

- json-server accepté pour mock API
- Pas de MySQL, PostgreSQL ou autre SGBD
- Focus sur le front-end



Versioning avec GitHub

Git et GitHub sont des outils indispensables dans le développement professionnel. Votre projet doit être versionné dès le premier jour avec des commits réguliers qui racontent l'histoire de votre développement.

Dépôt créé dès le jour 1

Initialisez votre repository GitHub immédiatement et configurez les accès pour tous les membres de l'équipe.

Commits réguliers

Minimum 1 commit par membre par jour avec des messages clairs et descriptifs suivant les conventions définies ensemble.

Branches recommandées

Utilisez des feature branches pour développer chaque fonctionnalité isolément avant de merger sur main.

README.md complet

Documentez votre projet avec des instructions d'installation, d'utilisation et une description claire.

🟢 ACCESSIBLE

Projets accessibles

Quiz interactif multithématique

Inspirations : Kculture, Maître des fleurs, ...

Persistance : Meilleurs scores dans localStorage

Bonus : Timer par question, système de vies, progression sauvegardée entre sessions

Jeu de memory avec thèmes

Inspiration : Le jeu de société de base

Persistance : Meilleurs temps par difficulté

Bonus : Mode multijoueur local, thèmes personnalisables (animaux, drapeaux, emojis)

🟠 INTERMÉDIAIRE

Projets intermédiaires

Idle/incremental game

Inspiration : Cookie Clicker, Unboxathon, ...

Persistance : Gains, améliorations et dernière connexion/présence du joueur

Bonus : Système de gain hors-ligne, système de renaissance (rebirth)

Tetris moderne

Inspiration : Tetris 🤡, Tricky Tower, ...

Persistance : High scores et paramètres de jeu (vitesse, thème)

Bonus : Niveaux de difficulté progressifs, power-ups temporaires, thèmes visuels multiples

🔴 AVANCÉ

Projets avancés

Tower defense

Inspirations : Artisan TD, Plant VS Zombie, ...

Persistance : Progression de campagne, argent accumulé, tours débloquées

Bonus : Améliorations/enchantement de tourelles, fast-forward avec récompenses

Party game (local, sauf bonus)

Inspirations : Mario Party, Pummel Party, ...

Persistance : Données de la partie en cours (objets, étoiles/objectif, ...)

Bonus : Synchronisation WebSocket (🤯) pour du multi-joueurs sur plusieurs périphériques

Note de projet (collective)

30%

Architecture POO

Qualité de l'héritage, du polymorphisme et de l'encapsulation

25%

Gestion d'erreurs

Robustesse du code et traitement des cas limites

20%

UI/UX Responsive

Qualité de l'interface et adaptation multi-écrans

15%

Versioning Git

Régularité des commits et qualité de la documentation

10%

Créativité

Originalité du projet et fonctionnalités bonus

+5%

Accessibilité

Accessibilité HTML et CSS valide (critères RGAA/WCAG)

Note de présentation (individuelle)

Voici la répartition des points pour la note de présentation individuelle sur 20.

Participation orale	/6	Prise de parole claire et audible, engagement actif.
Maîtrise du sujet	/5	Compréhension technique approfondie des parties du projet.
Clarté d'explication	/4	Capacité à vulgariser des concepts complexes et à démontrer.
Réponse aux questions	/3	Pertinence, précision et réactivité face aux interrogations.
Professionnalisme	/2	Posture, gestion efficace du temps et qualité des supports utilisés.

Modalités de présentation

Chaque groupe disposera d'un maximum de 15 minutes environ, réparties en trois phases distinctes, pour présenter son travail et répondre aux questions. Il est essentiel de respecter ce timing pour assurer le bon déroulement de l'évaluation pour tous les participants.

Présentation du projet

Les 5 premières minutes seront consacrées à la démonstration de votre projet. Mettez en avant les fonctionnalités principales, les choix techniques majeurs et les aspects innovants de votre solution.

Questions de la classe

La classe aura ensuite 5 minutes pour poser des questions sur votre projet. Préparez-vous à expliquer votre code, justifier vos décisions de conception et partager votre expérience.

Questions du formateur

Deux à trois questions techniques individuelles du formateur. Cela permettra d'approfondir certains points spécifiques de votre implémentation et d'évaluer votre compréhension des concepts abordés.

Consignes de présentation

Principe fondamental : Pitch investisseur

Chaque membre de l'équipe doit prendre la parole et participer activement à la présentation. C'est une occasion d'exercer vos compétences en communication et de démontrer votre maîtrise du projet dans son ensemble.

> **Règle d'or :** Dans le monde professionnel, si un associé reste silencieux face aux investisseurs, il perd toute crédibilité. Résultat : il est souvent écarté du projet dès la première levée de fonds. Ici, la présentation est une simulation de cette réalité. Votre engagement oral est donc primordial.

Conséquences sur l'évaluation

- La note globale du projet, reflétant la qualité technique et l'innovation, est commune à tous les membres de l'équipe. Elle représente le succès collectif de votre travail.
- La note de présentation est individuelle. Elle évalue votre capacité à communiquer clairement, à défendre vos choix et à interagir avec le public.
- Un membre qui ne participe pas activement à la présentation verra sa note individuelle impactée, même si le projet technique est excellent. La participation orale et la capacité à "vendre" son projet sont des compétences clés qui seront évaluées.

Conseils pour votre présentation

1. Introduction (30-45s)

- Nom du projet
- Composition de l'équipe et rôles
- Pitch concis : "Notre projet permet de..."

2. Démonstration Live (2min)

- Mettez en lumière le produit en conditions réelles.
- Parcourez l'expérience utilisateur complète.
- Présentez les fonctionnalités principales de manière fluide.
- En cas de bug, restez professionnel et expliquez le comportement attendu.

3. Aspects Techniques (1m30)

Répartissez les explications entre les membres de l'équipe :

- Architecture des classes et héritages
- Gestion des erreurs et cas limites
- Responsive design et choix techniques

4. Difficultés & Apprentissages (45s)

- Décrivez le principal défi rencontré.
- Expliquez la solution trouvée ou le compromis accepté.
- Partagez ce que vous avez appris de cette expérience.

5. Ouverture (15s)

- Mentionnez les évolutions possibles du projet.
- Indiquez ce que vous auriez développé avec plus de temps.

Gamma est très puissant pour générer un support de présentation à partir de prises de note. Même s'il n'est pas magique, il peut vous faire gagner un temps considérable.

Phase questions de la classe

Après la présentation de votre projet, la classe aura l'opportunité de poser des questions.

Objectifs de cette phase

Comprendre les choix techniques

Les questions visent à éclaircir les décisions techniques que vous avez prises, les algorithmes utilisés et l'architecture générale de votre solution.

Challenger les décisions d'architecture

Soyez prêts à défendre vos choix en matière d'architecture logicielle et de design, et à expliquer pourquoi certaines approches ont été privilégiées.

Proposer des améliorations

Certaines questions pourront vous amener à réfléchir sur des pistes d'amélioration ou des fonctionnalités futures non envisagées initialement.

Conseils aux présentateurs durant la phase de questions

Écoutez attentivement

Laissez l'interlocuteur terminer sa question avant de formuler votre réponse. Cela montre du respect et assure une compréhension complète.

Reformulez si nécessaire

Si la question n'est pas claire, n'hésitez pas à la reformuler avec vos propres mots pour vous assurer d'y répondre précisément.

Répondez honnêtement

Il est préférable d'admettre "on n'y a pas pensé" plutôt que d'inventer une réponse vague.

Respectez la prise de parole

Ne vous coupez pas la parole entre membres de l'équipe. Désignez un porte-parole ou laissez celui qui est le plus à même de répondre s'exprimer.

Questions techniques individuelles

Après la présentation de votre projet et la phase de questions de la classe, chaque membre de l'équipe sera soumis à une série de 2 à 3 questions techniques individuelles.

Exemples de questions :

Sur le code

- « Montre-moi la classe X, explique son rôle. »
- « Pourquoi as-tu utilisé un héritage ici ? »
- « Comment fonctionne ce try/catch ? »

Sur les choix techniques

- « Pourquoi localStorage plutôt qu'un backend ? »
- « Comment le responsive est géré sur cet écran ? »
- « Quel est le lien entre ces classes ? »

Sur le processus

- « Quelle partie du code as-tu développée ? »
- « Quel a été ton principal obstacle ? »
- « Si tu devais refaire ce projet, que changerais-tu ? »

En équipe, même si on ne travaille que sur une seule partie, on se doit de **comprendre le travail des autres**. Communiquez entre vous !

Conseils

#1 : Commencez simple

Ne voyez pas trop grand dès le départ. Créez d'abord un MVP fonctionnel avec les features essentielles, puis ajoutez progressivement des améliorations. Un projet simple mais bien fait vaut mieux qu'un projet ambitieux inachevé.

#2 : Testez régulièrement

Testez votre code à chaque modification majeure sur différents navigateurs et appareils. Utilisez les DevTools pour déboguer et optimiser. La détection précoce des bugs vous fera gagner un temps précieux.

#3 : Documentez au fur et à mesure

N'attendez pas la fin pour documenter votre code. Écrivez des commentaires clairs, maintenez votre README à jour et documentez vos choix d'architecture pendant le développement.

Ressources

Pour réussir cette gamejam, vous aurez besoin de maîtriser certains concepts et d'avoir accès aux bonnes ressources. Prenez le temps de les consulter avant de démarrer.

Documentation officielle

- MDN Web Docs pour JavaScript
- Documentation Canvas API
- Guide localStorage et sessionStorage
- Spécifications ES6+ classes
- WCAG (guidelines internationales) et RGAA (référentiel français)

Outils de développement

- VS Code avec extensions ESLint et Prettier
- DevTools pour debug
- Git
- Figma ou Excalidraw pour le maquettage

Frameworks CSS recommandés

- Tailwind CSS pour utility-first
- CSS Grid et Flexbox natifs
- Animations CSS3 pour l'interactivité

Préparation

Checklist finale avant démarrage du code

- ✅ Équipe constituée et rôles définis
- ✅ Idée de projet choisie et validée
- ✅ Repository GitHub créé et configuré
- ✅ Environnement de dev installé et testé
- ✅ Wireframes des écrans esquissés
- ✅ Architecture des classes planifiée
- ✅ Première tâche assignée à chaque membre

> **Derniers conseils avant de démarrer :** Organisez-vous en équipe, répartissez clairement les rôles, communiquez régulièrement sur l'avancement et n'hésitez pas à demander de l'aide.
> Cette gamejam est avant tout une expérience d'apprentissage collaborative autour de JavaScript.
>
> **Rappel : Vous n'êtes pas seul, je fais parti de chaque équipe en cas de besoin.**
>
> Bon courage et amusez-vous bien ! 🚀